

NMI Online Ordering Hard Cut

Summary

This change hard-cuts online-ordering payments from Dejavoo/iPOS to NMI for storefront checkout only.

In-store POS payment flows are intentionally untouched.

Key decisions:

- rollout is a hard cut, not dual-run
- credentials are merchant-scoped, not location-device-scoped
- synchronous NMI sale/void/refund responses are the source of truth
- reconciliation is supported as an on-demand admin action first
- the existing key names were preserved:
 - `nmiTokenizationKey`
 - `nmiPrivateApiKey`

What This Replaces

Old online-ordering payment path:

- Dejavoo/iPOS device selection
- TPN-based bootstrap
- whitelist-domain flow
- location payment device dependency for storefront checkout

New online-ordering payment path:

- merchant-level NMI credential record
- NMI browser tokenization key for checkout
- NMI private API key for server-side sale/void/refund
- no whitelist step
- no TPN/device routing in storefront checkout

What Was Implemented

1. Merchant credential model

Added:

- `public.merchant_payment_credentials`
- `public.merchant_payment_credential_access_log`

Added SQL RPCs:

- `list_merchant_payment_credentials(uuid)`
- `upsert_merchant_payment_credentials(uuid, text, text, text, boolean)`
- `get_merchant_payment_api_secret(uuid, text)`

File:

- `supabase/migrations/20260504123000_nmi_online_ordering_credentials.sql`

Design:

- `tokenization_key` is stored in-table
- `private_api_key_secret_id` points to Supabase Vault
- provider is constrained to `nmi`
- `unique (merchant_id, provider)`

Security correction applied:

- HQ/admin is the only authenticated role allowed to read or mutate merchant payment credentials directly
- merchant users are not allowed to read or rotate the private NMI API key through the new RPCs
- service-role callers are still allowed where server-side payment flows require it

2. Storefront payment bootstrap

The `process-online-payment` edge function was repurposed to return NMI bootstrap config instead of Dejavoo device/bootstrap fields.

New response contract:

- `success`
- `provider: 'nmi'`

- `tokenization_key`
- `merchant_payment_credential_id`

File:

- `supabase/functions/process-online-payment/index.ts`

3. Storefront card widget

The old client-side Dejavoo/FTD payment form was replaced with NMI React checkout.

Package added:

- `@nmipayments/nmi-pay-react`

Files:

- `package.json`
- `app/sites/components/checkout/PaymentCardForm.tsx`
- `app/sites/components/checkout/CheckoutPage.tsx`

Behavior:

- checkout requests NMI bootstrap config
- NMI component tokenizes card details in-browser
- checkout stores the latest `payment_token`
- order submission sends `payment_token`
- `payment_token_id` is still accepted as a legacy alias during cutover

4. Online order charge flow

The online-order edge function now charges NMI synchronously before final order creation.

Files:

- `supabase/functions/create-online-order/index.ts`
- `supabase/functions/_shared/nmi.ts`

Implemented:

- `POST /api/v5/payments/sale`
- uses merchant credential secret from Vault-backed RPC
- no longer depends on `payment_device_id`

- persists NMI results into generic `order_payments` fields

Persisted fields include:

- `processor_name = 'nmi'`
- `transaction_id`
- `authorization_code`
- `auth_code`
- `reference_number`
- `card_type`
- `card_last_four`
- `result_code`
- `result_message`
- `gateway_fee`
- `metadata.provider = 'nmi'`

5. Cancel, void, and refund flows

Implemented NMI server-side reversal paths.

Files:

- `supabase/functions/cancel-online-order/index.ts`
- `app/actions/orders/process-refund.ts`
- `app/actions/orders/void-order.ts`
- `lib/payments/nmi.ts`

Implemented endpoints:

- `POST /api/v5/payments/{transaction_id}/void`
- `POST /api/v5/payments/{transaction_id}/refund`
- `GET /api/v5/payments/{transaction_id}`

Behavior:

- unsettled payments use NMI void
- settled payments use NMI refund
- local payment/order state is updated after successful processor reversal

6. HQ admin settings

HQ online-store settings were updated to use NMI credentials instead of Dejavoo online-ordering fields.

Files:

- `app/manage/actions/admin-merchant/online-ordering.ts`
- `app/manage/merchants/[merchantId]/components/OnlineStoreTab.tsx`
- `lib/queries/use-admin-online-ordering.ts`
- `app/manage/merchants/[merchantId]/page.tsx`

Changed:

- active payment configuration fields are now:
 - NMI Tokenization Key
 - NMI Private API Key
- old Dejavoo whitelist and merchant-id controls were removed from the active online-store UI path
- online store enable flow now validates NMI configuration instead of Dejavoo readiness

7. Merchant dashboard cleanup

Merchant-facing online-ordering settings no longer carry Dejavoo/iPOS payment-device fields in their active data model.

Files:

- `app/dashboard/online-ordering/actions.ts`
- `app/dashboard/online-ordering/hooks/useOnlineOrderingSettings.ts`
- `app/dashboard/online-ordering/page.tsx`

Meaning:

- merchant dashboard still cannot edit payments
- payment credentials remain HQ-managed
- stale device/TPN fields were removed from the merchant storefront settings model

8. Reconciliation action

Added an on-demand HQ reconciliation action for NMI-backed order payments.

Files:

- `app/manage/actions/admin-merchant/online-payment-reconciliation.ts`
- `lib/queries/use-admin-online-ordering.ts`

Behavior:

- loads an NMI transaction by stored `transaction_id`
- updates local `order_payments` state from current processor status
- writes to credential access log

Removed or Disabled

- Dejavoo online-ordering bootstrap behavior
- Dejavoo whitelist requirement in the online checkout path
- location-device/TPN dependency for storefront card checkout
- orphaned old helper:
 - `supabase/functions/create-online-order/ipospays.ts`

Dejavoo terminal/in-store code outside online ordering was not changed.

Files Changed

Added

- `docs/HANDOFF-2026-05-04-NMI-ONLINE-ORDERING-HARD-CUT.md`
- `lib/payments/nmi.ts`
- `supabase/functions/_shared/nmi.ts`
- `supabase/migrations/20260504123000_nmi_online_ordering_credentials.sql`
- `app/manage/actions/admin-merchant/online-payment-reconciliation.ts`

Modified

- `app/actions/orders/process-refund.ts`
- `app/actions/orders/void-order.ts`
- `app/dashboard/online-ordering/actions.ts`
- `app/dashboard/online-ordering/hooks/useOnlineOrderingSettings.ts`
- `app/dashboard/online-ordering/page.tsx`
- `app/manage/actions/admin-merchant/online-ordering.ts`
- `app/manage/actions/upload-merchant-logo.ts`
- `app/manage/merchants/[merchantId]/components/OnlineStoreTab.tsx`

- `app/manage/merchants/[merchantId]/page.tsx`
- `app/sites/components/checkout/CheckoutPage.tsx`
- `app/sites/components/checkout/PaymentCardForm.tsx`
- `lib/queries/use-admin-online-ordering.ts`
- `package.json`
- `package-lock.json`
- `pnpm-lock.yaml`
- `supabase/functions/cancel-online-order/index.ts`
- `supabase/functions/create-online-order/index.ts`
- `supabase/functions/process-online-payment/index.ts`

Deleted

- `supabase/functions/create-online-order/ipospays.ts`

Validation Performed

- package install completed for `@nmipayments/nmi-pay-react`
- targeted TypeScript scan on the touched NMI/admin/storefront files returned no matches
- repo-wide `tsc` still has unrelated existing errors outside this workstream

Not performed yet:

- migration apply on staging
- edge-function deploy
- Next app redeploy
- real NMI sandbox checkout
- real refund/void test

Remaining Gaps

These are still open:

1. Scheduled reconciliation job
 - on-demand reconciliation exists
 - scheduled reconciliation is not implemented yet
2. Staging rollout

- migration still needs to be applied
- functions still need deployment
- app still needs redeploy

3. Runtime verification

- no real sandbox sale/decline/refund/void test has been executed yet

Rollout Steps

1. Apply:

- `supabase/migrations/20260504123000_nmi_online_ordering_credentials.sql`

2. Deploy edge functions:

- `process-online-payment`
- `create-online-order`
- `cancel-online-order`

3. Redeploy Next app

4. Configure HQ merchant NMI credentials

5. Run sandbox verification

Test Checklist

1. Admin save flow

- `save nmiTokenizationKey`
- `save nmiPrivateApiKey`
- confirm private key is not readable back

2. Storefront bootstrap

- call `process-online-payment`
- confirm:
 - `provider = 'nmi'`
 - `tokenization_key exists`
 - `merchant_payment_credential_id exists`

3. Checkout success

- `tokenize card successfully`
- `submit order`

- confirm order is created once
- `confirm order_payments.processor_name = 'nmi'`

4. Decline path

- declined card returns user-facing failure
- no order created

5. Cancel/void/refund

- unsettled cancel uses void
- settled reversal uses refund
- local order/payment state updates correctly

6. Regression

- cash-in-store still works
- inactive store still blocks payment bootstrap
- merchant dashboard still cannot edit payment credentials
- in-store POS payment flows remain untouched

Current Status

Implementation is largely complete for the hard cut.

Not ready to call fully done yet because:

- scheduled reconciliation is still pending
- nothing has been applied or tested on staging yet